

MEMORIA DE PRÁCTICAS

1^o Análisis de
Complejidad

Pablo Fazio Arrabal
2º Doble Grado Ing. Informática y
Matemáticas

ANÁLISIS DE COMPLEJIDAD EXPERIMENTAL

Desde el punto de vista teórico, el análisis de un algoritmo nos permite determinar cuál es su complejidad teórica. En la práctica, nos encontramos también con muchos casos en los que determinar esta complejidad de un programa complejo de forma teórica es difícil, o incluso imposible si no disponemos del código fuente. Aun así, la experimentación con distintos valores de entrada nos ofrece información sobre la posible complejidad de un programa. En esta práctica nos proponen diseñar un programa en Java para medir experimentalmente la complejidad de otro programa ya compilado.

Para ello, se nos dará una clase denominada `Algoritmo.class` para la cual debemos implementar una clase `Analizador.java` que sea capaz de determinar automáticamente el orden de complejidad de la función `f(long n)` contenida en `Algoritmo.class`. Dicho orden de complejidad será siempre uno de los siguientes tipos:

Complejidad	Salida java Analizador
$\Theta(1)$	1
$\Theta(\log n)$	LOGN
$\Theta(n)$	N
$\Theta(n \log(n))$	NLOGN
$\Theta(n^2)$	N2
$\Theta(n^3)$	N3
$\Theta(2^n)$	2N
$\Theta(n!)$	NF

Hablando ya sobre mi enfoque en este proyecto, he realizado la práctica junto a Nuria Pedrosa Ortigosa, alumna del Doble Grado de Ingeniería Informática y Matemáticas, en una resolución conjunta del planteamiento de la clase java `Analizador` y de las diferentes pruebas experimentales a través de las cuáles hemos probado nuestro código. Así pues, resumiré y comentaré mi código `Analizador.java` como también otras clases y programas que me han ayudado a debatir más los rangos de mi proyecto.

Debemos tener en cuenta que se nos proporciona al comenzar el proyecto una clase prueba `Algoritmo.class`, y dos ficheros `.java` del tipo `Analizador.java`, nuestra clase `Main` y `Temporizador.java`, el cuál actúa como un reloj que usamos para medir los tiempos de ejecución de `Algoritmo.class`.

`Analizador.java` trae un pequeño ejemplo donde se nos indica que la clase proporcionada es de orden de complejidad `N2`. Por último, nos indican dos estrategias principales a seguir para la resolución del ejercicio, nosotros hemos tomado una mayor aproximación hacia la Estrategia 1, así que, veamos por primera vez nuestra clase `Analizador.java`:

```

public class Analizador {

    private static int numIteraciones = 5;
    private static Temporizador temporizador = new Temporizador();

    public static void main(String[] args) {
        long n1 = 10;
        long n2 = 20;
        double ratio = mediaIntentos(n2)/mediaIntentos(n1);

        if (ratio>1000){
            if (ratio<3000){
                System.out.println("2N");
            } else{
                System.out.println("NF");
            }
        } else {
            n1 = 10000;
            n2 = 20000;
            ratio = mediaIntentos(n2)/mediaIntentos(n1);
            if (6 <= ratio && ratio < 10.0){
                System.out.println("N3");
            } else if (3.3 <= ratio && ratio < 6.0){
                System.out.println("N2");
            } else if (3<=ratio &&ratio<3.3){
                System.out.println("NLOGN");
            } else {
                n1 =1000;
                n2 =1000000;
                ratio=mediaIntentos(n2)/mediaIntentos(n1);
                if (ratio > 90){
                    System.out.println("N");
                } else if (ratio<=1.005) {
                    System.out.println("1");
                }else {
                    System.out.println("LOGN");
                }
            }
        }
    }

    private static double mediaIntentos(long num) {

        int media=0;

        for (int i = 0 ; i<numIteraciones ; i++){
            temporizador.reiniciar();
            temporizador.iniciar();
            Algoritmo.f(num);
            temporizador.parar();
            media += temporizador.tiempoPasado();
        }

        return media/(double)numIteraciones;
    }
}

```

En un resumen rápido del fichero Analizador.java se ve que se realiza un bucle con cinco iteraciones de nuestra función Algoritmo.f (num), calculando sus respectivos tiempos y se compara el ratio del orden de complejidad mediante la diferencia de la media de estos intentos en dos tamaños diferentes de entrada. (Estos tamaños podrán de ser de cualquier tipo, los dejados como ejemplo son orientativos, se realizan muchas más pruebas en otros).

Disecionemos un poco el recorrido que realiza nuestro programa:

```
private static int numIteraciones = 5;
private static Temporizador temporizador = new Temporizador();
```

En primer lugar, es claro ver que declara dos variables, una de tipo entero constante viendo el numero de iteraciones que se producirán en el bucle (puede ser un dato variable en función de si queremos ver más o menos casos) y por otro lado el constructor del objeto Temporizador del cuál usaremos sus métodos para hallar el tiempo que tarda en ejecutarse Algoritmo.class en f(long num).

```
public static void main(String[] args) {
    long n1 = 10;
    long n2 = 20;
    double ratio = mediaIntentos(n2)/mediaIntentos(n1);
```

Después entramos en la clase principal Main, en ella inicializamos dos variables de tipo long que representarán los tamaños de entrada que se resolverán en la clase Algoritmo, como hemos comentado, el valor de ellos y su tipo dependerá de la prueba que realicemos con ellos. Por otro lado se inicializa la variable double ratio que es calculada como la diferencia proporcional entre los dos tamaños en la clase mediaIntentos(long n).

```
private static double mediaIntentos(long num) {

    int media=0;

    for (int i = 0 ; i<numIteraciones ; i++){
        temporizador.reiniciar();
        temporizador.iniciar();
        Algoritmo.f(num);
        temporizador.parar();
        media += temporizador.tiempoPasado();
    }

    return media/(double)numIteraciones;
}
```

En tercer lugar, implementamos un método privado que realiza el cálculo del tiempo medio en numIteraciones intentos para un cierto tamaño dado. Se inicializa la media y después se recorre el bucle numIteraciones veces reiniciando nuevos tiempos para cada entrada y añadiendo el resultado a media. Una vez acabado el bucle, devolvemos el valor medio de las iteraciones.

```

if (ratio>1000){
    if (ratio<3000){
        System.out.println("2N");
    } else{
        System.out.println("NF");
    }
}

```

Tras encontrar el ratio gracias a nuestro método privado comenzamos viendo si pertenece a los valores exponencialmente grandes como son 2^n y $n!$. Es claro que, $2^n \in O(n!)$ por lo que es trivial que sea el ratio más grande de todos ellos, además el ratio de 2^n es claro que incluso para valores pequeños se vaya bastante rápido al infinito.

```

} else {
    n1 = 10000;
    n2 = 20000;
    ratio = mediaIntentos(n2)/mediaIntentos(n1);
    if (6 <= ratio && ratio < 10.0){
        System.out.println("N3");
    } else if (3.3 <= ratio && ratio < 6.0){
        System.out.println("N2");
    } else if (3<=ratio &&ratio<3.3){
        System.out.println("NLOGN");
    }
}

```

Después de ver si nuestro Algoritmo. Class tiene orden exponencial veamos que ocurre para ordenes casi lineales, volvemos a definir nuevos tamaños de entrada, aunque no es necesario, simplemente es cuestión de simplificación de los ratios, volvemos a usar el mismo procedimiento de antes por el cual crece de manera menor en los casos de n^2 , n^3 y $n \log n$.

```

} else {
    n1 =1000;
    n2 =1000000;
    ratio=mediaIntentos(n2)/mediaIntentos(n1);
    if (ratio > 90){
        System.out.println("N");
    } else if (ratio<=1.005) {
        System.out.println("1");
    }else {
        System.out.println("LOGN");
    }
}

```

Y ya por último, como en las anteriores lo comentábamos el caso del orden de complejidad lineal y constante. Es claro que los valores de los ratios en cada uno de estos intervalos se han realizado teniendo en cuenta el tamaño de entrada adquirido para ellos y el resultado en las diferentes pruebas por las que han pasado, por ello voy a presentar las distintas clases extra que he realizado y maneras que me han ayudado a darle un valor a cada una de estas funciones.

En primer lugar, he creado dos clases de prueba en java, PruebaAlgoritmo.java y Tiempos.java:

```
public class PruebaAlgoritmo {

    private static Temporizador temporizador = new Temporizador();

    public static void main(String[] args) {

        for(int i = 10; i<100; i++) {
            int suma=0;
            temporizador.reiniciar();
            temporizador.iniciar();
            Algoritmo.f(i);
            temporizador.parar();
            suma=(int) temporizador.tiempoPasado();
            System.out.println(suma);
        }

    }

}

public class Tiempos {

    public static void main(String[] args) {
        Temporizador temporizador = new Temporizador();
        int[] tamEntrada = {10,30,50,70,90,100,150,500};
        long[] tiempos1 = new long[tamEntrada.length];

        try (PrintWriter fich1 = new PrintWriter("algoritmo1.csv");
            ) {
            fich1.println("Tam,Tiempo");

            for (int i = 0; i < tamEntrada.length; i++) {

                temporizador.reiniciar();
                temporizador.iniciar();
                Algoritmo.f(tamEntrada[i]);
                temporizador.parar();

                tiempos1[i] = temporizador.tiempoPasado();
                fich1.print(tamEntrada[i]);
                fich1.print(",");
                fich1.println(tiempos1[i]);

            }

            System.out.println(Arrays.toString(tiempos1));

        } catch (FileNotFoundException e) {
            System.err.println("Fichero no se puede crear");
        }

    }

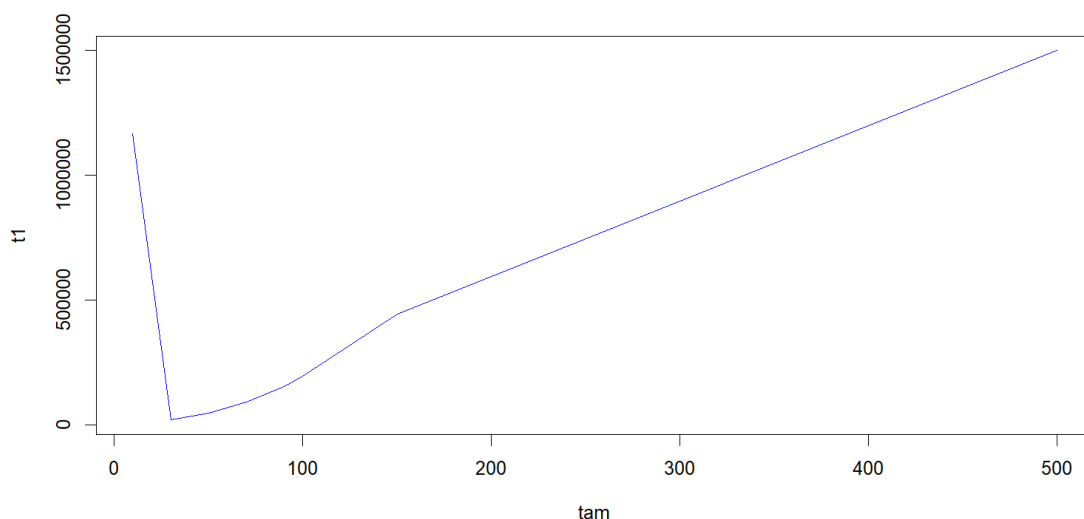
}
```

La explicación del primer método es bastante claro, un bucle con las iteraciones deseadas que expresa por pantalla el tiempo tardado en realizar `Algoritmo.f(long num)` para cada tamaño de entrada deseado y definido en el bucle, con ello podemos tener una idea de su crecimiento a lo largo de un intervalo de tamaños concreto, me ha ayudado sobre todo a comprobar intuitivamente la solución de los ejercicios una vez dado la repuesta sobre su orden de complejidad.

El segundo fichero necesita de mayor explicación, muy similar al modelo desarrollado en la primera práctica de laboratorio en clase, se escoge un array de diferentes tamaños variables según el usuario, el cual imprime su tamaño y posterior tiempo de ejecución del algoritmo, a su vez gracias a la opción `PrintWriter` desarrolla un fichero .csv para su posterior evaluación en otro programa, en el caso actual podemos ver que `algoritmo1.csv` es de la forma:

```
Tam, Tiempo
10,1164900
30,20200
50,48100
70,92700
90,155400
100,195300
150,444100
500,1498100
```

A través de este fichero incluso he usado el programa **R Studio** para hacer una comparación por gráficas sobre los diferentes `Algoritmo.class` dados, en este caso:



Es claro ver que crece casi linealmente incluso para valores pequeños de tamaños de entrada, es normal, también el desajuste de datos en el primer valor de entrada. Por tanto, hemos usado varias herramientas para encontrar el orden de complejidad de manera experimental de un programa ya compilado.

En mi conclusión , creo que ha sido una buena práctica para aprender sobre el orden de complejidad y comparación de algoritmos de una manera más práctica y he concluido que tenemos muchas herramientas a mano para trabajar todo ello, yo he trabajado en el entorno de desarrollo de **Eclipse**.